ArchiveWebShelf

A plain-language map of its addresses, server, files, and safety boundaries

In one sentence: the browser recorder saves a WACZ, the Python helper files it on this computer, and the bundled ReplayWeb.page reader opens it in a separate browser origin.

The trip from a live page to a saved replay

The main idea

ArchiveWebShelf is a local program made from a browser screen, a small Python server, a browser extension, and ReplayWeb.page reader files. The screen and extension talk to the helper through a private loopback address. The helper stores your archive files and folder list on disk.

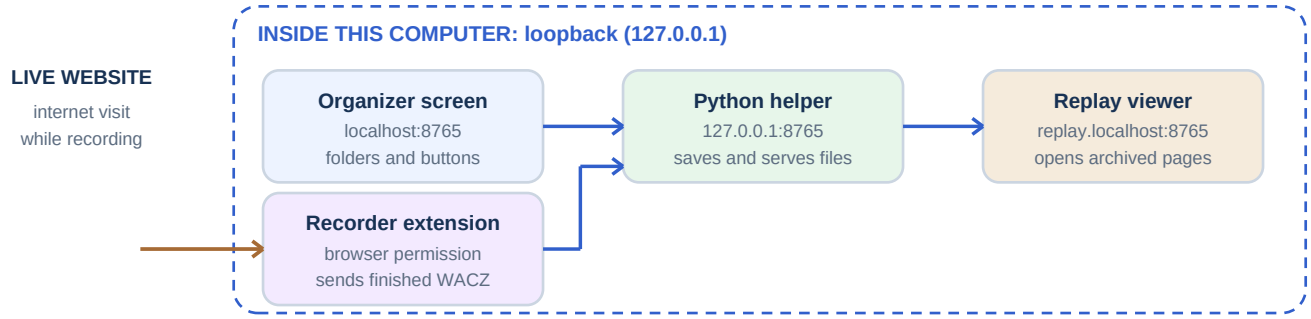
Local means the server accepts connections through this computer's own loopback address. It is not a server listening for other computers on your Wi-Fi.

This guide describes the ArchiveWebShelf files in the app folder as of October 21, 2026. If the code changes, the exact ports or checks may change too.

1. Three addresses, one computer

The addresses look different, but the server is running on the same PC. Think of the PC as a building, localhost as its street name, and the port number as a numbered door.

Address	Who uses it	Plain meaning
`http://localhost:8765`	Browser screen	Opens the organizer page and folder buttons.
`http://127.0.0.1:8765`	Recorder extension	The extension sends the finished archive to the local helper here.
`http://replay.localhost:8765`	Replay viewer	Opens the reader and archived pages in a different browser origin.



What the pieces of the address mean

- 127.0.0.1 is a special loopback IP address: traffic goes back into this same computer.
- localhost is the readable local name for this computer. On another PC, localhost would mean that other PC.
- 8765 is a port, like a numbered door. By default the server tries 8765, then 8766 through 8794 if needed. The extension checks the same range.
- The server binds only to 127.0.0.1. It does not bind to the computer's Wi-Fi or Ethernet address.

Why use three names?

Web browsers define a website origin using its protocol, host name, and port. So localhost and replay.localhost are separate browser origins even though both lead back to this PC. The Python helper is one process serving different routes depending on which host name was requested.

2. What runs and where the data goes

When you record, your browser still visits the real website. The extension watches the page and items that load while recording is on. It does not crawl every link by itself.

Step	What happens
1. Start	The Recorder connects to the active browser tab and starts capturing the pages and resources you visit.
2. Stop	The Recorder packages the captured material as a WACZ archive. A WACZ is a file that can contain saved pages, images, scripts, and an index.
3. File	The extension sends the WACZ to the Python helper at 127.0.0.1. The helper stores it under library/ and adds its folder and name to archive-shelf-data.json.
4. View	The screen opens the bundled ReplayWeb.page reader at replay.localhost. The reader gets the chosen WACZ from the local library and replays its saved pages.

What needs the internet?

Action	Internet?	Why
Capture a live website	Yes	Your browser must reach the website you are saving.
Open an archive already in the Shelf	No, for the reader	The ReplayWeb.page UI and service worker are bundled locally, and the WACZ is in library/.
Check for a ReplayWeb.page update	Yes	The updater contacts the npm package registry when you run it.
Send a completed WACZ to ArchiveWebShelf	No	The transfer goes to 127.0.0.1 on this same computer, not to a cloud service.

ReplayWeb.page is a browser reader. Its frontend runs in the browser and its service worker helps replay saved URLs. The Python helper supplies the bundled files and the archive bytes. See the official embedding guide: https://replayweb.page/docs/embedding/.

3. Safety rules and their limits

These are the checks visible in this copy of the code. They reduce some common mistakes and browser-based access, but they do not turn the app into a password-protected network service.

Check or choice	What it does	What it does not do
Loopback binding	Server listens on 127.0.0.1 only, on ports 8765-8794 by default. Other computers cannot connect to that loopback address on your PC.	It does not block programs already running on your PC from making local requests.
Host and Origin checks	Checks which local name was used. Only the app host can change folders or archives; the replay host is read-only for organizer routes.	These headers are not a login. A local program can send or fake HTTP headers.
Separate replay origin	Archived pages are opened under replay.localhost, separate from the organizer's localhost origin. ReplayWeb.page's require-subdomain-iframe option is used for increased origin isolation.	It is not a virtual machine or a promise that untrusted archive content is harmless.
Content Security Policy	Limits what the organizer and viewer shell may load, such as scripts, frames, and connections.	It does not make every archived website safe or scan archives for malware.
File and size checks	Only catalogued archive IDs and names are served from library/. JSON requests are limited to 1 MiB; archive uploads default to 20 GiB maximum.	A very large accepted archive can still consume a lot of disk space.
Recorder permissions	The extension uses browser debugging and broad website access so it can capture the sites you choose to record. Cookie capture is off by default.	Those are powerful permissions. Install only the Recorder copy you trust, and opt into cookies only when needed.

Important hardening note: the current local API accepts any Origin that looks like a Chrome extension address, not only this Recorder's exact extension ID. The server remains loopback-only, but pinning the one intended extension ID would strengthen the check.

The server uses local HTTP, has no sign-in, and does not encrypt the files on disk. Keep the app folder on a trusted Windows account. Do not change the server to listen on all network interfaces or expose the port through a router. Treat archives from unknown sources as untrusted content.

4. How to explain it to someone else

ArchiveWebShelf runs a small Python helper that listens only to my own computer. The Recorder captures pages I visit and sends a WACZ file to that helper. It saves the file and its folder information locally. A bundled ReplayWeb.page reader opens it on a separate local browser address so archived pages are separated from the organizer screen.

Easy safety checklist

- Keep the black server window open while using ArchiveWebShelf. Closing it stops the helper.
- Do not expose ports 8765-8794 to Wi-Fi or the internet.
- Only install the Recorder extension from your own trusted project folder.
- Leave cookie capture off unless you deliberately need it. Cookies may contain login information.
- Back up library/ and archive-shelf-data.json together. Those hold the archives and their folder list.
- Remember: local-only is not the same as encrypted or password-protected, and origin separation is not an antivirus scan.

Where to look in the code

File or folder	What it explains
organizer_server.py	Loopback binding, port selection, Host and Origin checks, API routes, upload limits, archive serving.
index.html	Organizer screen and how it sends the selected archive to the viewer.
replay-viewer.html and replay-viewer.js	The isolated ReplayWeb.page view and archive URL.
browser-extension/Archive Shelf Recorder/manifest.json	Permissions used by the ready-to-load browser extension.
browser-extension/archiveweb-page-source/	Readable Recorder source, including its permission list and capture code.

Official ReplayWeb.page reference: https://replayweb.page/docs/embedding/ - documents the service worker, embed setup, and requireSubdomainIframe option. This guide summarizes the local ArchiveWebShelf code; it is not a security audit or a guarantee of safety.